TimeTensors: Forecasting Benchmark

Experiment Guideline

Gaspard Berthelier

9 August 2026

Abstract

TimeTensors studies how data selection, sampling, normalization, optimization loss, model class, and training scope affect time-series forecasts. This document defines the forecasting task, the seven implemented experiment families, and the exact common protocol used by their launchers.

Contents

1 Time-series forecasting task

Let $\mathcal{U} = \{0, \dots, U-1\}$ index series (called *users* in the code), let $d \in \{1, \dots, D\}$ index variables within a series, and let $t \in \{0, \dots, T-1\}$ index ordered dates. The complete panel is

$$Z = (z_{u,d,t}) \in \mathbb{R}^{U \times D \times T}.$$

The benchmark CSV loader treats each non-date column as one series, hence all publication datasets use $D = 1$; the implementation nevertheless retains the general D-variate tensor contract.

Fix a lookback length $L \geq 1$ and a forecast horizon $H \geq 1$. A query date t is the last observed date. Its input and target are

$$X_{t,u} = Z_{u,:,(t-L,t]} = (z_{u,:,t-L+1}, \dots, z_{u,:,t}) \in \mathbb{R}^{D \times L}, \tag{1}$$

$$Y_{t,u} = Z_{u,:,(t,t+H]} = (z_{u,:,t+1}, \dots, z_{u,:,t+H}) \in \mathbb{R}^{D \times H}. \tag{2}$$

Thus L is the number of available past values and H the number of future values predicted at once. A forecasting model is a map

$$F_\theta : \mathbb{R}^{D \times L} \longrightarrow \mathbb{R}^{D \times H}, \qquad \widehat{Y}_{t,u} = F_\theta(X_{t,u}),$$

where θ contains trainable parameters for learned models and is empty or frozen for persistence and Chronos-2.

The dates are split chronologically into target-date blocks $S_{\text{tr}}, S_{\text{va}}, S_{\text{te}}$ in proportions 0.6/0.2/0.2. For any block S, the admissible queries are

$$\mathcal{T}(S) = \{t : L-1 \le t \le T-H-1,\ \{t+1,\dots,t+H\} \subseteq S\}.$$

The full target therefore belongs to its split, while the lookback may cross the preceding split boundary. In particular, changing L does not change the validation or test target dates. The default training and evaluation pair sets are $\mathcal{A}_S = \mathcal{U} \times \mathcal{T}(S)$, before the filters and strides defined below.

For a lookback X, statistics are computed independently along its time axis:

$$\mu_X = \frac{1}{L}\sum_{\ell=1}^{L} X_{:,\ell}, \qquad \sigma_X = \left(\frac{1}{L}\sum_{\ell=1}^{L}(X_{:,\ell}-\mu_X)^2\right)^{1/2}, \qquad \varepsilon = 10^{-8}.$$

The population standard deviation is used. Model normalization is inverted before the loss is evaluated, so predictions and targets passed to every loss are in the original data space.

2 Experiment 1: constant windows and users

For the univariate benchmark, an accessible pair (u,t) is valid when its lookback is finite and non-constant:

$$a_{u,t} = \mathbf{1}\left[X_{t,u} \text{ is finite and } \sigma_{X_{t,u}} > 0\right].$$

Window filtering removes only pairs for which $a_{u,t} = 0$. User filtering is more restrictive: for split S it retains

$$\mathcal{U}_S^{\text{safe}} = \{u \in \mathcal{U} : a_{u,t} = 1 \text{ for every } t \in \mathcal{T}(S)\}.$$

In the general D-variate implementation, window filtering accepts a user–date pair if at least one variable is finite and non-constant, whereas user filtering drops a user if any accessible variable-window is constant or non-finite. These definitions coincide for the $D = 1$ benchmark tensors.

The launcher compares exactly five policies in full and ultra: keep all pairs; `remove_train_windows`, `remove_eval_windows`, or `remove_all_windows`; and `drop_all_users`. Train and evaluation interventions are deliberately separated: train-only filtering tests an optimization effect, whereas evaluation filtering changes the measured population. Curated source exclusions from each dataset's `config.json` are applied before these dynamic policies and are not an experimental factor.

The family uses instance-normalized PatchTST in the full profile and adds DLinear in ultra. Its MSE tables should be interpreted with the retained numbers of users and windows, since lower error after evaluation filtering is not by itself evidence of a better forecaster.

3 Experiment 2: sampling and batch size

The sampler controls which accessible pairs form optimizer batches. With the current default of one user per dataset item, its four modes are:

- `random`: the single dataset item draws a query date and then one user uniformly at random;
- `dates`: item i fixes the ith accessible date and draws one user at random;
- `individuals`: item i fixes user i and draws one accessible date at random;
- `all`: items deterministically enumerate every accessible (u,t) pair in date-major order.

After filtering, let N_m be the dataset length in mode m. Ignoring removed pairs,

$$N_{\text{random}} = 1, \qquad N_{\text{dates}} = |\mathcal{T}(S_{\text{tr}})|, \qquad N_{\text{individuals}} = U, \qquad N_{\text{all}} = |\mathcal{A}_{S_{\text{tr}}}|.$$

For nominal batch size B and configured epoch count E, the current training loop performs

$$K_m = E\left\lceil \frac{N_m}{B} \right\rceil$$

optimizer steps. Consequently, the full-profile value $E = 10{,}000$ is exactly 10,000 optimizer steps only in random mode; the modes do not share a fixed update budget. Moreover, random mode returns one actual window per batch with the current dataset length, irrespective of nominal B. This behavior is part of the present implementation and must be accounted for when interpreting the batch-size sweep.

Full and ultra cross the four modes with $B \in \{64, 256, 1024\}$. Evaluation is held to exhaustive pair enumeration with stride H; changing B changes its batching but not its query set or metric. The model uses instance normalization, and the shared model axis is PatchTST in full and PatchTST plus DLinear in ultra.

4 Experiment 3: normalization layers

The wrapper applies an invertible normalization N before the base forecaster f_θ and restores its output afterward:

$$\widehat{Y} = N^{-1}(f_\theta(N(X)))\,.$$

Let $(\mu_0, \sigma_0, a_0, b_0)$ be the scalar mean, standard deviation, minimum, and maximum computed over all accessible training lookbacks. Let $a_X = \min_\ell X_{:,\ell}$ and $b_X = \max_\ell X_{:,\ell}$. The sweep compares:

$$\begin{aligned}
N_{\mathrm{id}}(X) &= X, \\
N_{\mathrm{std}}(X) &= \frac{X-\mu_0}{\sigma_0+\varepsilon}, & N^{-1}_{\mathrm{std}}(Q) &= Q(\sigma_0+\varepsilon)+\mu_0, \\
N_{\mathrm{mm}}(X) &= \frac{X-a_0}{b_0-a_0+\varepsilon}, & N^{-1}_{\mathrm{mm}}(Q) &= Q(b_0-a_0+\varepsilon)+a_0, \\
N_{\mathrm{imm},X}(X) &= \frac{X-a_X}{b_X-a_X+\varepsilon}, & N^{-1}_{\mathrm{imm},X}(Q) &= Q(b_X-a_X+\varepsilon)+a_X, \\
N_{\mathrm{inst},X}(X) &= \frac{X-\mu_X}{\sigma_X+\varepsilon}, & N^{-1}_{\mathrm{inst},X}(Q) &= Q(\sigma_X+\varepsilon)+\mu_X.
\end{aligned}$$

The code names are respectively `identity`, `standard`, `min-max`, `in-min-max`, and `instance`. The sixth method, `revin`, adds one learnable scale and bias per variable:

$$N_{\mathrm{RevIN},X}(X) = \gamma \odot N_{\mathrm{inst},X}(X) + \beta, \quad N^{-1}_{\mathrm{RevIN},X}(Q) = \frac{Q-\beta}{\gamma+\varepsilon}(\sigma_X+\varepsilon)+\mu_X.$$

Thus `instance` and `revin` use the same `RevIN` class, with affine parameters disabled and enabled, respectively. All instance statistics are detached and cached from the current lookback. The experiment fixes the training loss to nMSE and reports inverse-transformed test MSE separately for each shared backbone.

5 Experiment 4: reference orders of error

This experiment establishes the error scale before architecture-specific ablations. It compares three qualitatively different forecasters under the same split and evaluation queries:

$$\begin{aligned}
\widehat{Y}^{\mathrm{pers}}_{t,u,:,h} &= X_{t,u,:,L}, && h = 1,\dots,H, \\
\widehat{Y}^{\mathrm{PatchTST}}_{t,u} &= F_{\widehat{\theta}}(X_{t,u}), \\
\widehat{Y}^{\mathrm{Chronos}}_{t,u} &= F^{\mathrm{median}}_{\theta_0}(X_{t,u}).
\end{aligned}$$

Persistence has no trainable parameters. PatchTST is trained from scratch with instance normalization and nMSE. Chronos-2 is loaded from the local checkpoint, kept frozen, uses identity normalization and no cross-learning, and returns the median forecast quantile. Users with an accessible constant lookback are dropped in both training and evaluation by default.

The family writes a combined test MSE table and a combined worst-user-decile MSE table. The comparison is an order-of-error reference, not a claim that the three methods have equal training data or compute.

6 Experiment 5: training losses

For one prediction, let $Q = DH$ and let j index its $D \times H$ elements. The five implemented training objectives are

$$\mathrm{MSE}(\widehat{Y}, Y) = \frac{1}{Q}\sum_{j=1}^{Q}(\widehat{Y}_j - Y_j)^2, \tag{3}$$

$$\mathrm{MAE}(\widehat{Y}, Y) = \frac{1}{Q}\sum_{j=1}^{Q}|\widehat{Y}_j - Y_j|, \tag{4}$$

$$\mathrm{nMSE}(\widehat{Y}, Y \mid X) = \frac{1}{Q}\sum_{j=1}^{Q}\left(\frac{\widehat{Y}_j - Y_j}{\sigma_X + \varepsilon}\right)^2, \tag{5}$$

$$\mathrm{nMAE}(\widehat{Y}, Y \mid X) = \frac{1}{Q}\sum_{j=1}^{Q}\frac{|\widehat{Y}_j - Y_j|}{\sigma_X + \varepsilon}, \tag{6}$$

$$\mathrm{rMSE}(\widehat{Y}, Y \mid X) = \frac{1}{Q}\sum_{j=1}^{Q}\left(\frac{\widehat{Y}_j - Y_j}{|\mu_X| + \varepsilon}\right)^2. \tag{7}$$

The scales broadcast from each sample and variable over its horizon. The code name for the last objective is `relative_mse`. Because scaling is applied to prediction and target before the base MSE/MAE, these equations match the implementation exactly.

The sweep holds instance normalization fixed and changes only the training criterion. Its tables compare every run using data-space test MSE, so training curve magnitudes from different objectives are not directly comparable. Complete run artifacts additionally retain MSE, nMSE, MAE, nMAE, relative MSE, and per-user losses.

7 Experiment 6: linear models

Linear controls test how much performance follows from autoregressive structure alone. The unrestricted trainable model is

$$\mathrm{vec}(\widehat{Y}) = W\,\mathrm{vec}(X) + b, \qquad W \in \mathbb{R}^{DH \times DL}, \quad b \in \mathbb{R}^{DH}.$$

The periodic model uses period $p = \min(L, 168)$. With zero-based lag and horizon indices, define

$$I_h = \{\ell \in \{0, \dots, L-1\} : \ell \bmod p = (L + h) \bmod p\}.$$

It fits one head per horizon,

$$\widehat{Y}_{:,h} = W_h\,\mathrm{vec}(X_{:,I_h}) + b_h, \qquad h = 0, \dots, H-1,$$

so each forecast step sees only past positions of the same phase. No limit is placed on the number of previous cycles.

The `linear` and `periodic_linear` models are optimized by the common PyTorch training loop. `sklinear` instead deterministically unrolls every sampler-accessible training pair and fits scikit-learn multi-output ordinary least squares with an intercept. Persistence is included as a parameter-free reference. Each fitted linear method is evaluated with `identity`, `standard`, and non-affine `instance` normalization; the launcher also saves its coefficient tensor and plots. The test profile narrows this to `linear` and `sklinear` with instance normalization. All methods share one MSE table.

8 Experiment 7: central and per-user forecasting

A central model fits one parameter vector using all training users. Per-user training creates an independent model and output directory for each user in the training split, restricts every available split to that user, and then merges the resulting losses. For PatchTST this compares shared training with specialization. For frozen Chronos-2, ultra mode instead compares central inference with `cross_learning=true` against separate per-user inference with `cross_learning=false`.

Let n_u be the number of evaluated scalar errors for user u and let $\ell_{u,i}$ denote one such error. Define

$$\bar{\ell}_u = \frac{1}{n_u}\sum_{i=1}^{n_u}\ell_{u,i}, \qquad M_{\mathrm{elem}} = \frac{\sum_u\sum_i \ell_{u,i}}{\sum_u n_u}, \qquad M_{\mathrm{user}} = \frac{1}{U}\sum_u \bar{\ell}_u.$$

For $q = \max(1, \lceil 0.1U \rceil)$ and Top_q the users with the q largest means, the tail metric is

$$\mathrm{W10}(\ell) = \frac{1}{q} \sum_{u \in \mathrm{Top}_q} \bar{\ell}_u.$$

The current central artifacts store elementwise MSE, hence their reported `mse` reduces to M_{elem}. The per-user merger explicitly overwrites `mse` with M_{user}. The combined MSE table therefore uses these two different weightings, while the W10 table uses the same equal-user tail definition for both scopes. This is the implemented artifact contract and should be stated whenever the combined table is interpreted.

9 Experimental details

9.1 Datasets and task grid

Every CSV column other than `date` becomes one univariate series. The configured publication datasets are listed in Table 1; counts are after the curated source exclusions in the current `config.json` files.

Table 1: Configured publication datasets. Solar and Weather are resampled to hourly cadence during tensor construction.

Dataset	Series	Raw cadence	Used cadence	Construction
ETTh1	7	1 hour	1 hour	All seven non-date variables.
Electricity	312	1 hour	1 hour	321 raw series; nine configured exclusions.
Traffic	862	1 hour	1 hour	No configured source exclusion.
Solar	137	10 minutes	1 hour	Hourly sums.
Weather	21	10 minutes	1 hour	Hourly means.
Exchange Rate	8	1 day	1 day	No configured source exclusion.

Full and ultra use

$$(L, H) \in \{(168, 24), (336, 48), (504, 168)\}$$

and seeds $\{1, 2, 3\}$. Test mode uses only Electricity at $(504, 168)$ with seed 1. A dataset's sibling `config.json` is loaded during tensor construction: portable top-level options are applied first, the `timetensors` object overrides them, explicit run values override both, and all `drop_users` lists are merged additively.

9.2 Models

Table 2: Models exercised by the seven publication launchers.

Code name	Implemented forecast	Fit in this project
`persistence`	Repeats the last lookback value for all H steps.	None
`linear`	One affine map $\mathbb{R}^{DL} \to \mathbb{R}^{DH}$.	Adam
`periodic_linear`	One affine head per horizon using phase-matched lags, $p = \min(L, 168)$.	Adam
`sklinear`	Multi-output scikit-learn `LinearRegression` over all accessible windows.	Closed-form OLS
`dlinear`	Moving-average decomposition (kernel 25) followed by separate per-variable seasonal and trend maps $\mathbb{R}^L \to \mathbb{R}^H$.	Adam
`patchtst`	Patches of length 16 and stride 8; three encoder layers, 16 heads, $d_{\mathrm{model}} = 128$, $d_{\mathrm{ff}} = 256$, learned positional encoding, and a flattened forecast head.	Adam
`chronos`	Local frozen Chronos-2 pipeline; the wrapper selects the median quantile and optionally enables cross-learning.	Frozen

PatchTST uses attention dropout 0, encoder dropout 0.2, feed-forward/head dropout 0.2/0, GELU activations, and residual attention. DLinear and PatchTST form the shared model axis: full uses PatchTST and ultra adds DLinear. The reference and linear experiments use their fixed method lists instead. The central/per-user family uses PatchTST in test and full, and adds Chronos-2 in ultra.

9.3 Optimization, evaluation, and outputs

Unless a family overrides a value, training uses Adam with PyTorch defaults, learning rate 10^{-5}, batch size 256, random training sampling with stride 1, exhaustive evaluation sampling with stride H, and no scheduler, early stopping, or gradient clipping. Test runs $E = 20$ epochs and validates/logs every 10 optimizer steps. Full and ultra run $E = 10{,}000$ epochs and validate/log every 1,000 optimizer steps. As derived in Section 3, E counts full passes over the configured sampler and is not generally the optimizer-step count.

Non-filtering families default to dropping users with an accessible constant or non-finite lookback in both training and evaluation. The constants family sets those controls per policy. The reference family exposes independent overrides for its PatchTST choices. Chronos-2 and persistence set $E = 0$ and are evaluated without optimization.

Evaluation saves elementwise MSE, nMSE, MAE, nMAE, and relative MSE on each selected split, together with losses grouped by user, equal-user summaries, and W10 summaries. Tables use `test1`, average each metric tensor within a seed, then report the mean and sample standard deviation across seeds. Each row uses an explicit $\times 10^m$ scale. Every seed directory contains model, history, metadata, and loss artifacts.

9.4 Result identities and manifests

Every family continues as `outputs/family/dataset/L_H/backbone/configs/run_n/seed_n`. The ordered model-config folders are: `policy` for constants; `sampling_mode/batch_size` for sampling; `normalization` for normalizations; `loss` for losses; `normalization/loss` for the reference family; `normalization` with the linear method as backbone for linear models; and `scope` for central/per-user. No synthetic upper/lower folder level is required.

Training budget, optimizer, split, strides, and logging/plot controls are pipeline configs in `manifest.json`; scheduler placement is runtime-only. One seed fixes all stochasticity. `EXPERIMENT_MODE` selects path subsets and is never a phase, family, path component, or computation-signature field.

Run signatures use only the schema, identity/model and pipeline/experiment configs, and seeds. Files are not fingerprinted for reuse; provenance is ignored and code/data reruns are manual. Only a global contract break changes the schema; `new` creates an unchanged-parameter repeat.

The current contract uses `schema_version: 1` and five states. The default `overwrite_exact` policy skips exact completion, resumes exact interruption, and allocates a new `run_n` for changed pipeline config. Finished seeds and runs remain `ready`; overall completion occurs only on the owning Slurm workflow's final successful exit. Completed manifests are authoritative, without later file hashing or synchronization checks. Tables support distinct/latest/average config handling and selected/latest/distinct/average repeat selection; explicit pipeline filters select configurations and must match even with one run. `SELECTED_RUNS.txt` stores exact-repeat selection and each report stores requested filters and obtained input manifests. Old synchronized outputs lacked the loss tensors needed for faithful migration and are isolated under `outputs/archive/legacy_pre_schema_v1_2026-08-07`.

After successful completion, each root workflow submits `publish.slurm` with an `afterok` dependency. It commits only the producer's exact stdout/stderr and launch-tagged run/report roots, excluding `*.pt`, `*.npy`, and `*.cbm`, then sources `$HOME/proxy.sh` with the ignored two-line `.secrets/proxy.credentials` and pushes `origin/main`. It never pulls or opens a pull request. `PUBLISH_RESULTS=false` disables submission; `-job-id` supports a manual retry.

The synthetic smoke explicitly enumerates every expected method in all seven families and requires seeds 1 and 2 for each. Its current run tree is below `outputs/synthetic_smoke/family/synthetic_smoke/24_6/numpy_linear_proxy/method/run_n/seed_n`; reports use `outputs/reports/synthetic_smoke`.

9.5 Main implementation classes

The implemented forward path can therefore be summarized as

$$(Z, S, L, H) \xrightarrow{\text{dataset/sampler}} (X, Y) \xrightarrow{\text{normalize, forecast, invert}} (\widehat{Y}, Y) \xrightarrow{\text{loss/evaluation}} \text{seed artifacts and tables.}$$

Table 3: Principal classes and their current responsibilities.

Class	Responsibility
`TimeSeriesData`	Stores Z, dates, stable user IDs, optional covariates, and the target dates owned by a split.
`IndexSampler`	Builds admissible query dates, applies stride/subsets and pair filtering, and implements the four sampling modes.
`TimeSeriesDataset`	Materializes $X_{t,u}$, $Y_{t,u}$, covariates, query IDs, user IDs, and window metadata from a sampled pair.
`TimeTensorModel`	Composes normalization, optional covariate augmentation, a base forecaster, inverse normalization, and optional output rules.
`LossWrapper`	Applies MSE or MAE after optional lookback-standard-deviation or absolute-lookback-mean scaling.
`TorchLearner`	Performs one optimizer update per loader batch, records interval-mean training loss, validates at step frequencies, and returns elementwise evaluation losses.
`SkLinearForecaster`	Deterministically unrolls accessible windows, applies the same reversible normalization contract, and fits/predicts with scikit-learn OLS.